

Predicting the price of a house based on its attribute

by: Nguyen Vinh Khang (101682945)

1.Introduction

As house pricing is going up every year, it makes it difficult to know whether the house price is a reasonable one. There are a lot of factors that determine the price of a house such as square feet, bedrooms, bathrooms, the year the house was built,.... Housing crisis can also drive the price up significantly. In Australia, the housing crisis has driven the price and rental price of a house up which has made thousands of people homeless [1]. Therefore, this project is to ascertain whether we can predict the price of the house based on given data to make better informed choices.

The paper is structured as follows: section 2 concerns the formulation of the project, section 3 concerns the methods used, section 4 is about the results of the models, section 5 is about the conclusion and the final section is the sources and the appendix.

2. Project formulation

The purpose of this project is to predict the price of a house based on the attributes of the house itself. In this project, the project will use the USA house prices dataset by Firat Özcan from Kaggle [2]. The machine learning task in the project is supervised learning, as it will use labels with features to train the model. There are a few unnecessary columns in the datasets, so the datasets will first be modified such that each data point contains the price, number of bedrooms and bathrooms, the area of the living space, lot, above and basement, the number of floors, and the condition of the house. Each data point represents the statistics of a single home in the US such as the price, the number of bedrooms and bathrooms. For the label, it will be the price of the house, as it is the objective of the project. The rest of the data will be the features. Most of the data in the dataset is continuous data. Only the condition variable is categorical data as it can only be an integer from 1 to 5 representing the condition of the house.

3. Method

3.1 Dataset structures

In the dataset, there will be 4140 data points representing 4140 houses. There are 18 columns representing different attributes in the datasets, but only 8 columns are needed. Therefore, the dataset needs to be cleaned up before training the models. There is no missing data in the dataset, so the dataset does not have to be modified any further. There are 17 features in the original dataset. However, there is no concrete evidence that the variables are directly connected to the price of the house, as it can depend on external factors such as where the house is located, the housing market which is difficult to determine how much they will affect the price without analyzing very deeply. As a result, the problem will become more complex, so I will simplify the problem by choosing 8 variables that have an effect on the price in real life [Appendix]. After looking at the scatter plot of each variable, three variables are removed as they do not seem to be linearly related to the price. The

rest of the variables either seem to be linearly related to the price and have a reasonable effect on the price of the house in real life **[Appendix]**. All of the variables data is already in the original dataset which makes it easy to fetch.

To confirm whether the model is able to predict the price of a house, there must be a validation set to validate the hypothesis. As there are 4140 data points in the dataset which is a relatively small dataset, I will use K-fold Cross validation. K-fold Cross validation is a good way to split the dataset as it is the most basic form of splitting, and it also helps avoid unlucky spots in the data, and estimate loss better **[3]**. K-fold Cross validation is to split the data into K subsets and use one subset for validation and the rest for training **[3]**. We repeat the process k times so that every subset is chosen for validation once **[3]**. I will split the dataset to 60% for training, 20% for validation, 20% for testing, as it is convenient for using $k = 4$.

3.2 Models

a. Polynomial regression

To predict the price which could vary widely, the models need to be a regression model. Although the variables chosen seem to be linearly related to the label, there may be non-linear patterns in the dataset. So, Polynomial regression seems to be the most suitable choice for the project, because it is able to demonstrate both the non-linear and linear pattern in the data and fit the data more accurately. The polynomial regression can be done by inserting a feature vector in a linear regression model **[3]**. The model can be easily implemented using `PolynomialFeatures()` and `LinearRegression()` from `sklearn`.

The polynomial regression model uses the mean squared error function, or the mean absolute error function for its loss function. The mean absolute error function is more suitable to the model, as the price can be very large making the errors quite large. Mean squared error is very sensitive to large numbers, so it is not very useful. For large prediction errors, the mean absolute error function resembles the Huber loss function more which makes it more accurate than the mean squared loss function **[3]**. The mean absolute error function can be imported from `sklearn` as `mean_absolute_error()`.

b. Deep learning (Multi-layer Perceptron)

As the problem can be non-linear, we can use models that can accommodate the non-linearity. Deep learning models are suitable as they have the ability to map the problem using the signal flow chart. Using the non-linear activation function, the hypothesis space of the model will be the same as linear regression or polynomial regression. **[3]** The project will be using Multi-layer Perceptron which is a simple deep learning model. It uses signal-flow charts or neural networks in the model. This means that it takes in a layer of inputs and modifies them in a hidden layer or hidden layers using weights to return the specified output. **[3]** The model will try to find the best weights to return the closest output to the label. This model can be implemented using `MLPRegressor` from `sklearn`.

To validate the model, mean absolute error will also be used as the loss function. As the hypothesis space of the model is the same as polynomial

regression. As mentioned before, mean absolute error is the most suitable due to the large prices making mean squared error not very reliable. The mean absolute error function can be imported from sklearn as `mean_absolute_error()` as mentioned before.

4. Results

To validate and optimize the model, we have to find the best degree based on the validation errors. After training the model, the results from polynomial regression are:

degrees	1	2	3	4	5
tr_errors	186993	185244	188080	189909	180383
val_errors	187346	187119	196407	217105	298195
degrees	6	7	8	9	10
tr_errors	185833	224341	212672	240164	231779
degrees	375210	642143	2235337	13428525	5589924

At first glance, the model seems to be less accurate as the degree becomes larger. The difference between the training error and the validation error gets bigger as the degree gets larger. Comparing the training errors and validation errors from various degrees, it seems that the second degrees result in the best prediction, as both of the errors of the second degree are the lowest, which is 187119 dollars.

To validate and optimize the multi-layer perceptron, we have to find how many layers the model should use based on the validation errors. The results from multi-layer perceptron are:

layers	1	2	3	4	5
tr_errors	191040	191283	191211	191203	191170
val_errors	191077	191307	191228	191165	191130
layers	6	7	8	9	10
tr_errors	191080	190154	191077	191307	190910
val_errors	191115	190161	191629	190823	190905

Looking at the result, the errors seem to be consistent despite the layers. The difference between the training error and the validation error is very small. Comparing the results, seven layers seem to be the best option that return the lowest error which is 190161 dollars.

Comparing the polynomial regression model and the multi-layer perceptron model, polynomial regression degree 2 seems to be better than multi-layer perceptron. However, the difference between the models is not much. Therefore, the final model chosen is the polynomial regression degree 2 model. To test the effectiveness of the model, the model needs to be tested with data not used before. we can use the test portion of the dataset mentioned in section 3.1. It is 20% of the original dataset, and it is constructed using the `train_test_split` function from `sklearn`. Using the test portions, we get the test error of the polynomial regression degree 2 is 172307 dollars. The multi - layer perceptron is also used to predict the test data. The test error from it is 178359. It confirms that polynomial regression can predict the data better.

5. Conclusion

The polynomial regression seems to be better at predicting than the multi - layer perceptron. However, the multi - layer perceptron model is not much behind the polynomial regression. From the test errors, both of the models are still very inaccurate. Both of the models seem to be underfitting as both of the training errors and validation errors are large for both models. Polynomial regression becomes less accurate as its degree gets larger, while Multi - layer perceptron remains consistent despite the number of layers changing. As the errors are almost 200000 dollars, this means that some of the features do not really correlate with the price of the house. Although using real life knowledge seems to be logical, it results in less accuracy for predicting data. The polynomial regression model seems to be optimal, but the multi - layer perceptron model may not be optimal as the number of neurons per layer can be changed to improve the accuracy but not by much. To further improve the machine learning method, the project needs more data, as the original data is pretty small and the models can increase the accuracy with more data. The features also need to be reselected for better correlation to the price. From there, more detailed analysis can be carried out to assess the problem better.

6. Sources

1. Duncan A Rouch(2023), *Fixing Australia's housing crisis: how to do better*, ReserchGate, Available at:
https://www.researchgate.net/publication/352371223_Fixing_Australia%27s_housing_crisis_how_to_do_better (Accessed 17/09/2024)
- 2.Firat Özcan (2022), *USA house prices*, Kaggle. Available at:
<https://www.kaggle.com/datasets/fratzcan/usa-house-prices?select=USA+Housing+Dataset.csv> (Accessed 17/09/2024)
- 3.A. Jung, "Machine Learning: The Basics", Springer, Singapore, 2022

Appendix

I code the notebook from scratch. However, I do look at the course assignment and previous projects in the course to see how some of the functions work, or how they are used.

Appendix A

October 4, 2024

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.neural_network import MLPRegressor
```

```
[2]: df = pd.read_csv('USA Housing Dataset.csv') #read the data from
      ↪file
df.info()
```

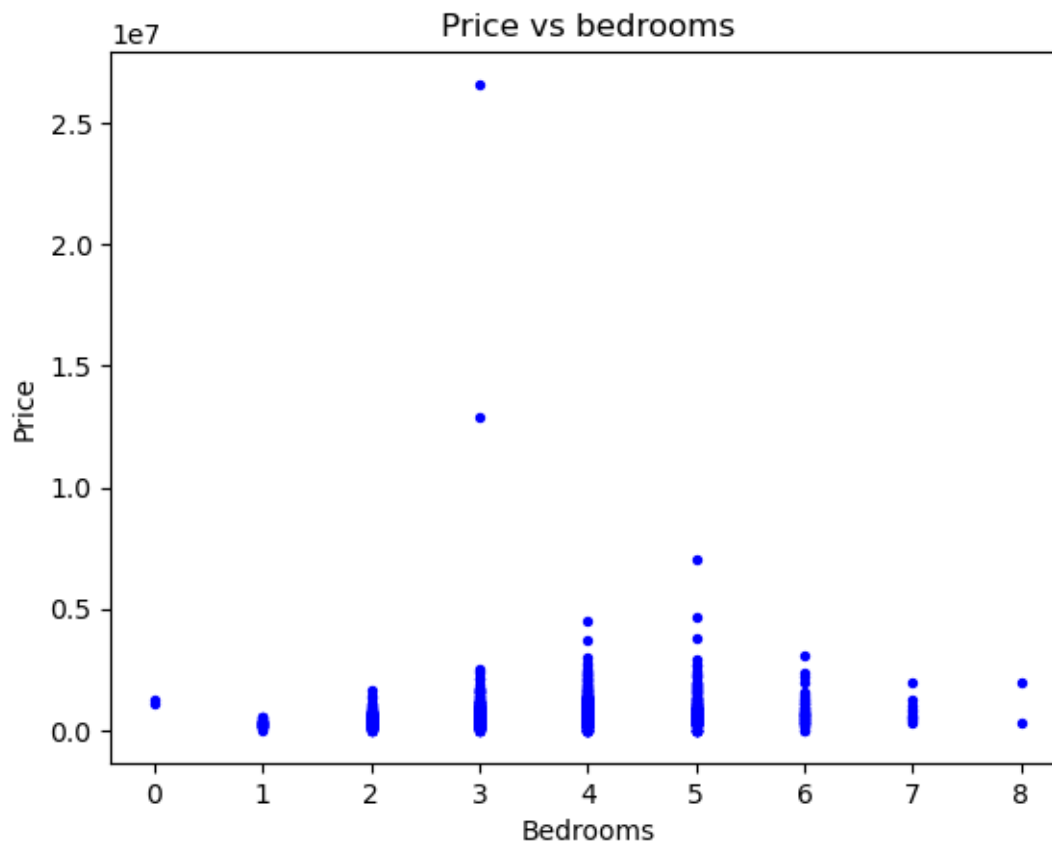
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4140 entries, 0 to 4139
Data columns (total 18 columns):
#   Column          Non-Null Count  Dtype
---  -
0   date            4140 non-null   object
1   price           4140 non-null   float64
2   bedrooms        4140 non-null   float64
3   bathrooms       4140 non-null   float64
4   sqft_living     4140 non-null   int64
5   sqft_lot        4140 non-null   int64
6   floors          4140 non-null   float64
7   waterfront      4140 non-null   int64
8   view            4140 non-null   int64
9   condition       4140 non-null   int64
10  sqft_above      4140 non-null   int64
11  sqft_basement   4140 non-null   int64
12  yr_built        4140 non-null   int64
13  yr_renovated    4140 non-null   int64
14  street          4140 non-null   object
15  city            4140 non-null   object
16  statezip        4140 non-null   object
17  country         4140 non-null   object
dtypes: float64(4), int64(9), object(5)
```

memory usage: 582.3+ KB

```
[3]: # I choose 8 variables which are 2,3,4,5,6,9,10,11 from the list above to plot
      ↪and see how they are
      # connected to the price
      fig, ax1 = plt.subplots()

      # create a scatter plot of datapoints
      # each datapoint is depicted by a dot in color 'blue' and size '10'
      ax1.scatter(df['bedrooms'], df['price'], color='b', s=8, label='datapoints from
      ↪the dataframe "newdata"')
      ax1.set(xlabel='Bedrooms', ylabel='Price') #define the axes
      ax1.set_title('Price vs bedrooms') # define the title of the plot

      fig.show() # display the plot on the screen
```

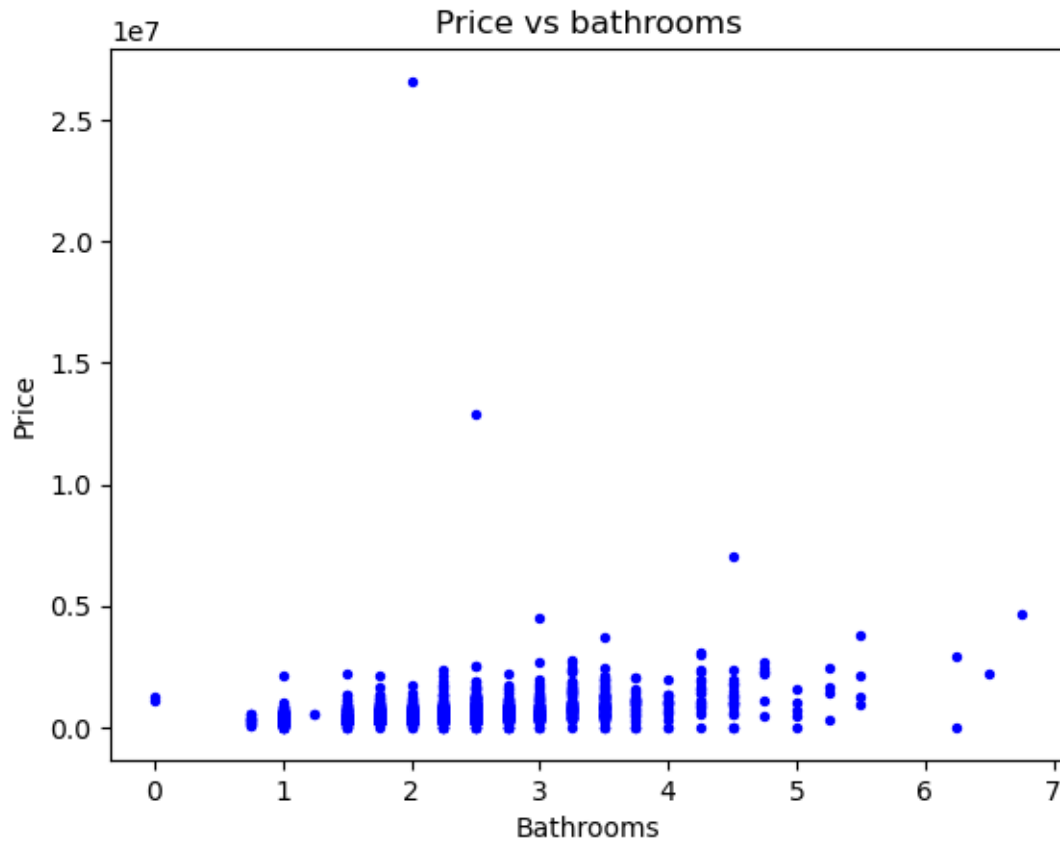


```
[4]: fig, ax2 = plt.subplots()

      ax2.scatter(df['bathrooms'], df['price'], color='b', s=8, label='datapoints
      ↪from the dataframe "newdata"')
```

```
ax2.set(xlabel='Bathrooms', ylabel='Price')
ax2.set_title('Price vs bathrooms')

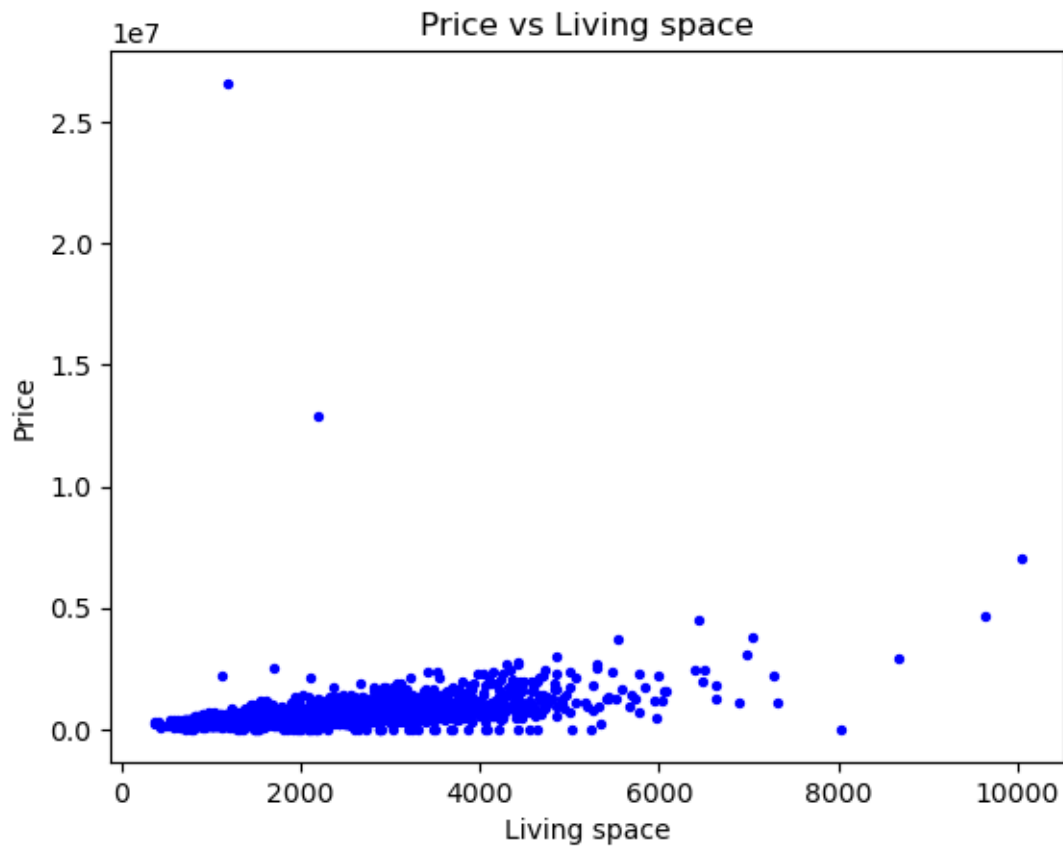
fig.show()
```



```
[5]: fig, ax3 = plt.subplots()

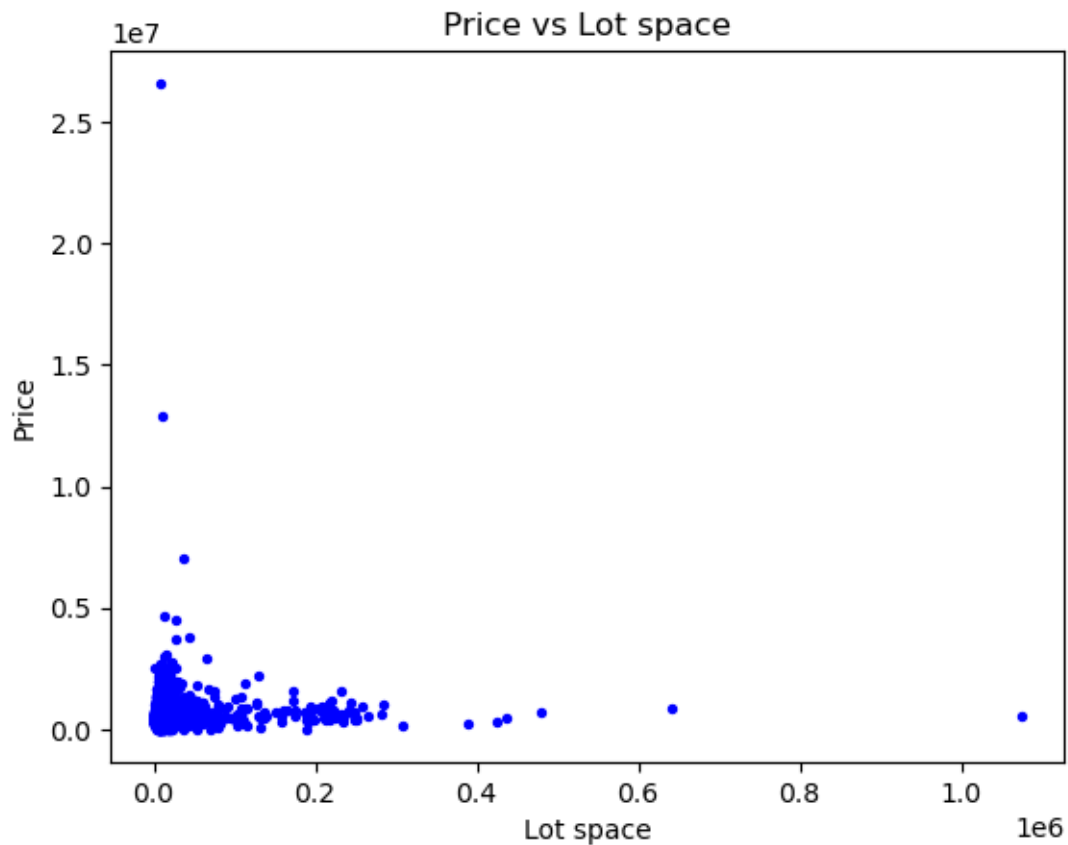
ax3.scatter(df['sqft_living'], df['price'], color='b', s=8, label='datapoints_
↳from the dataframe "newdata"')
ax3.set(xlabel='Living space', ylabel='Price')
ax3.set_title('Price vs Living space')

fig.show()
```



```
[6]: fig, ax4 = plt.subplots()

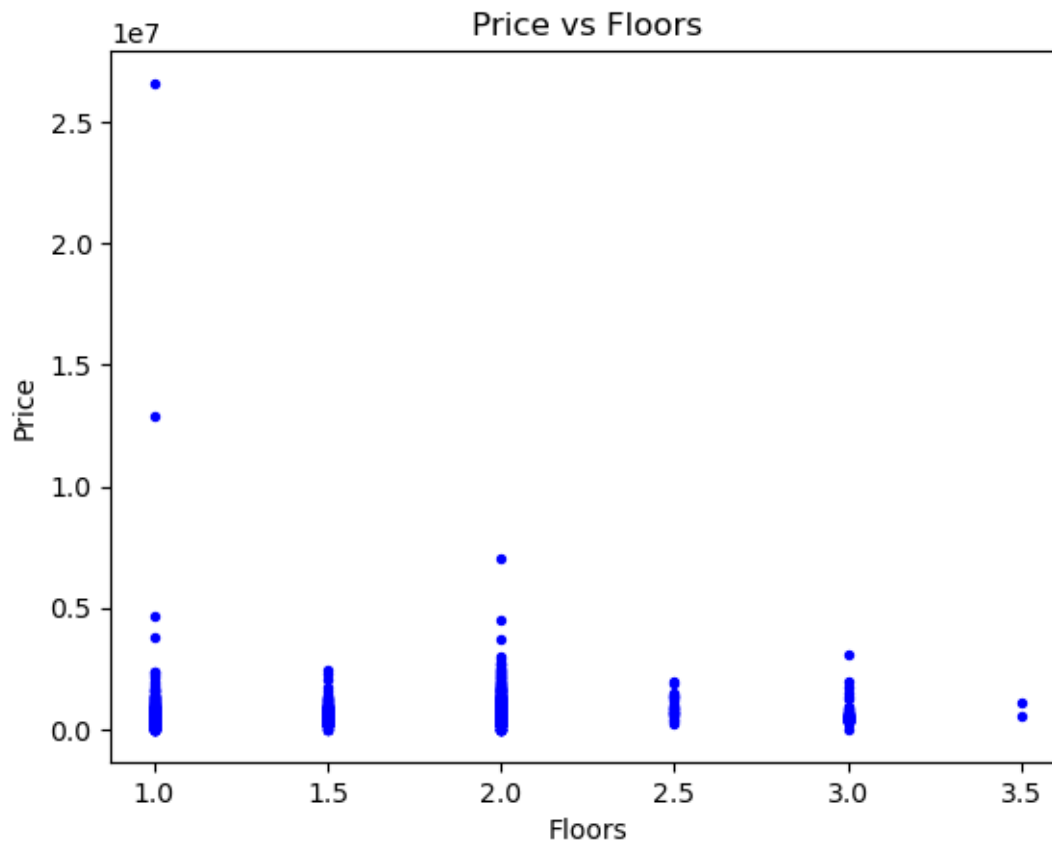
ax4.scatter(df['sqft_lot'], df['price'], color='b', s=8, label='datapoints from_
↳the dataframe "newdata"')
ax4.set(xlabel='Lot space', ylabel='Price')
ax4.set_title('Price vs Lot space')
fig.show()
```



```
[7]: fig, ax5 = plt.subplots()

ax5.scatter(df['floors'], df['price'], color='b', s=8, label='datapoints from_
↳the dataframe "newdata"')
ax5.set_xlabel='Floors', ylabel='Price')
ax5.set_title('Price vs Floors')

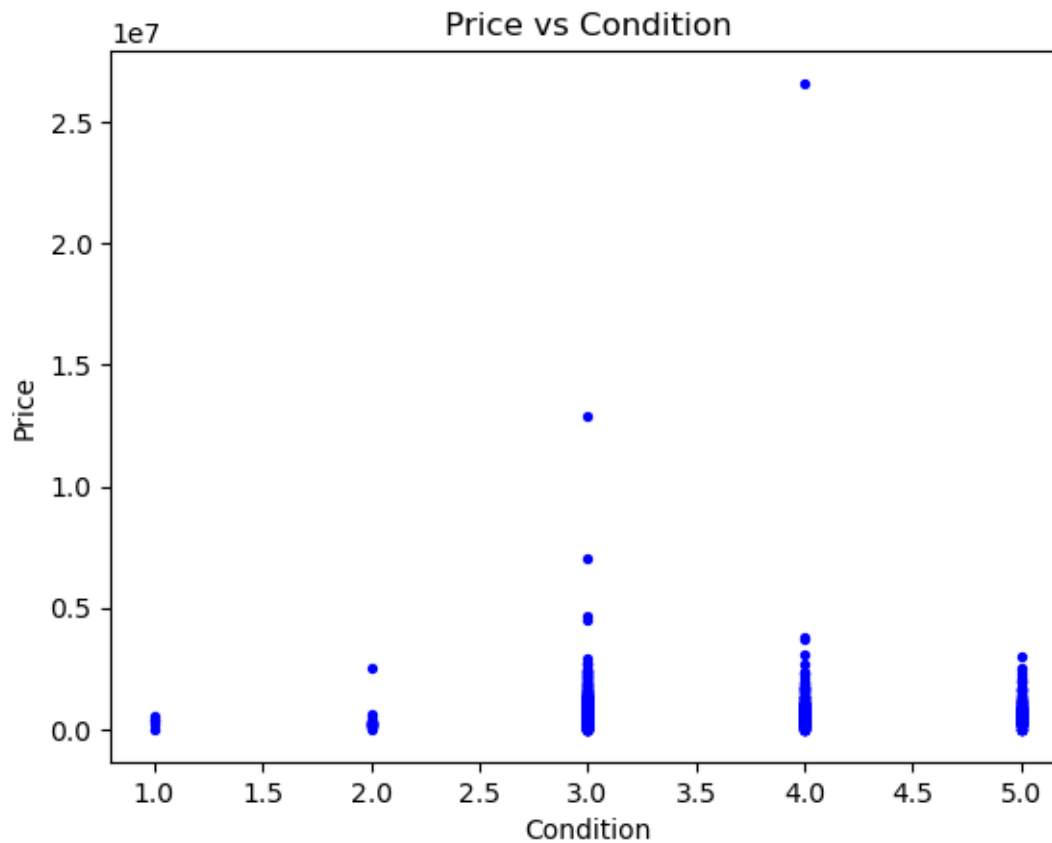
fig.show()
```



```
[8]: fig, ax6 = plt.subplots()

ax6.scatter(df['condition'], df['price'], color='b', s=8, label='datapoints_
↳from the dataframe "newdata"')
ax6.set_xlabel('Condition', ylabel='Price')
ax6.set_title('Price vs Condition')

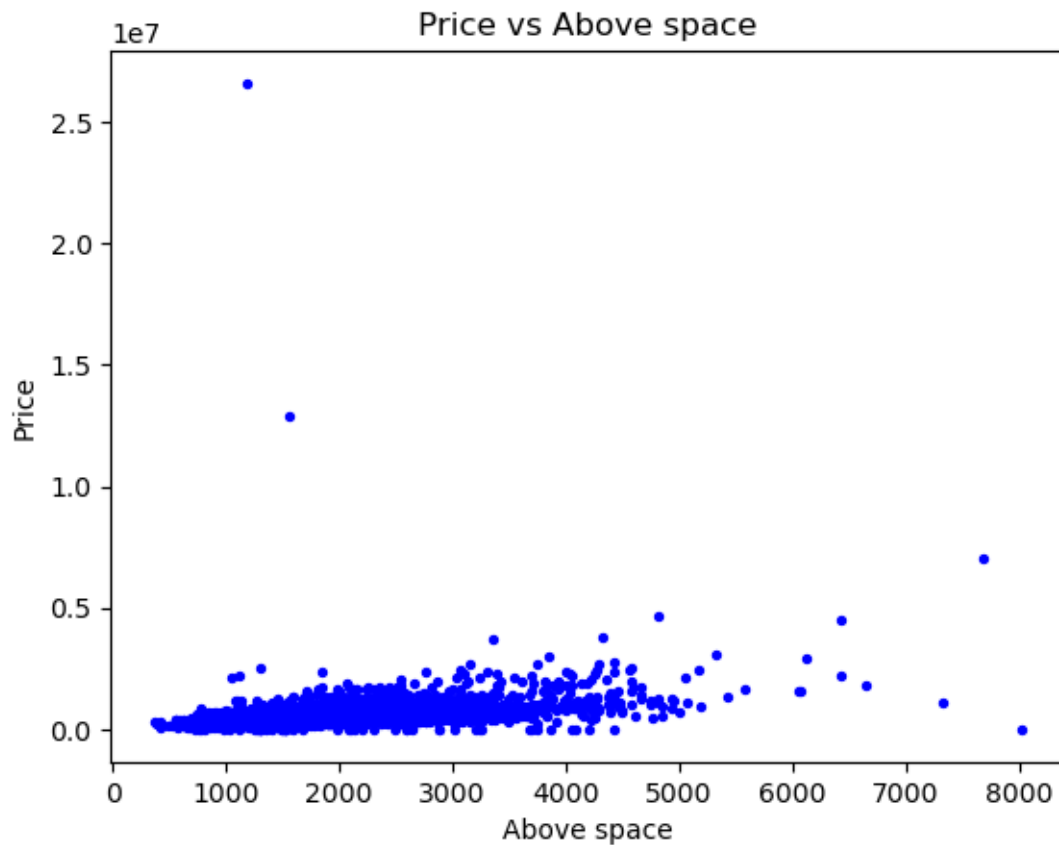
fig.show()
```



```
[9]: fig, ax7 = plt.subplots()

ax7.scatter(df['sqft_above'], df['price'], color='b', s=8, label='datapoints_
↳from the dataframe "newdata"')
ax7.set_xlabel('Above space', ylabel='Price')
ax7.set_title('Price vs Above space')

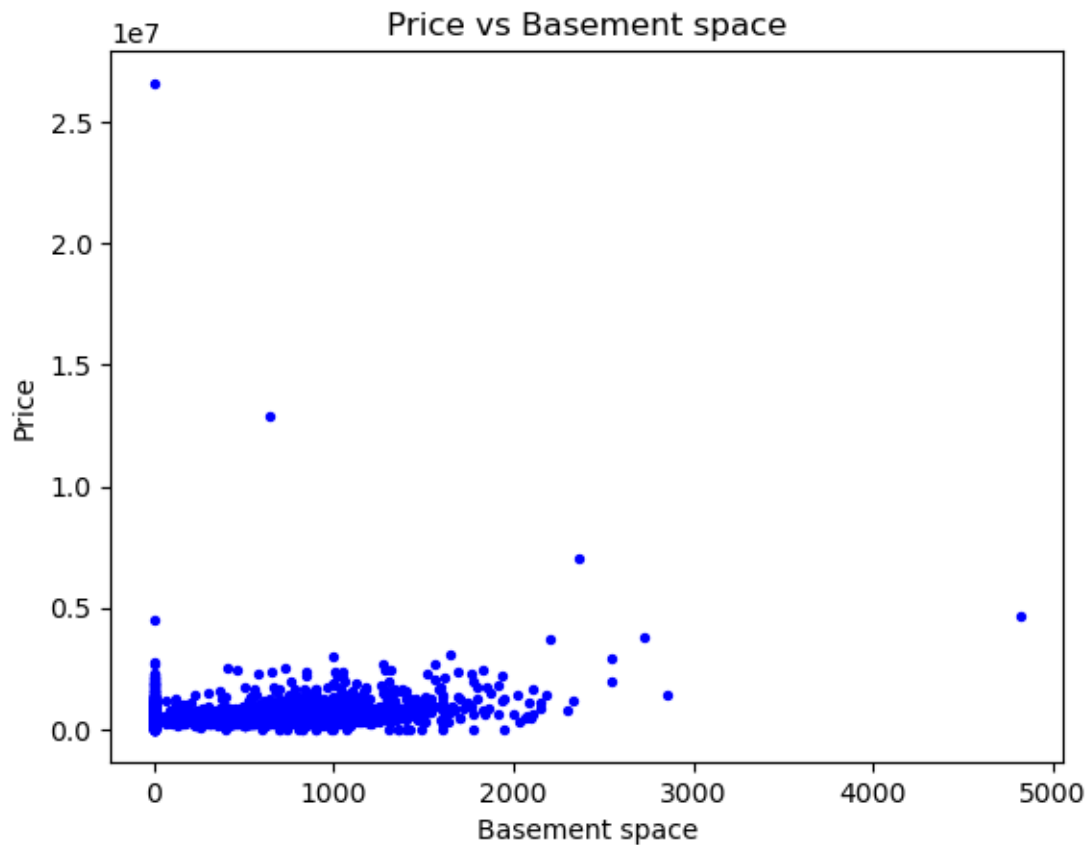
fig.show()
```



```
[10]: fig, ax8 = plt.subplots()

ax8.scatter(df['sqft_basement'], df['price'], color='b', s=8, label='datapoints_
↳from the dataframe "newdata"')
ax8.set_xlabel='Basement space', ylabel='Price')
ax8.set_title('Price vs Basement space')

fig.show()
```



```
[11]: data = df.drop(['date', 'waterfront', 'view', 'yr_built', 'yr_renovated',
                    'street', 'city', 'statezip', 'country', 'sqft_lot', 'floors',
                    ↪ 'condition'],axis=1)      #clean the data for necessary data
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4140 entries, 0 to 4139
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   price            4140 non-null   float64
1   bedrooms         4140 non-null   float64
2   bathrooms        4140 non-null   float64
3   sqft_living      4140 non-null   int64
4   sqft_above       4140 non-null   int64
5   sqft_basement    4140 non-null   int64
dtypes: float64(3), int64(3)
memory usage: 194.2 KB
```

```
[12]: X = data.drop(['price'], axis=1).to_numpy().reshape(-1,5)           #Choosing
      ↪ features and label
      y = data['price'].to_numpy()
```

```
[13]: kfold = KFold(n_splits=4, shuffle=True, random_state= 43)         #Declare
      ↪ kfold for later use

X_folds, X_test, y_folds, y_test = train_test_split(X, y, test_size= 0.2
      ↪ ,random_state= 101682945) #split the test data from the dataset.
```

```
[14]: pol_tr_errors = []
      pol_val_errors = []

      lin_regr = LinearRegression()

      for degree in range(1,11):                                         #choosing degrees for
      ↪ Polynomial Regression
          poly = PolynomialFeatures(degree=degree)
          tr_error = 0.0
          val_error = 0.0
          for i, (train, val) in enumerate(kfold.split(X_folds)):         #KFold
      ↪ validation and choosing validation set and training set
              X_train, y_train = X_folds[train], y_folds[train]
              X_val, y_val = X_folds[val], y_folds[val]

              X_train_poly = poly.fit_transform(X_train)                 #Training the
      ↪ model
              lin_regr.fit(X_train_poly, y_train)
              y_train_pred = lin_regr.predict(X_train_poly)
              tr_error += mean_absolute_error(y_train, y_train_pred)

              X_val_poly = poly.fit_transform(X_val)                     #Validationg the
      ↪ model
              y_val_pred = lin_regr.predict(X_val_poly)
              val_error += mean_absolute_error(y_val, y_val_pred)

              pol_tr_errors.append(tr_error / 4)                         #Average of
      ↪ the error estimate from KFold validation
              pol_val_errors.append(val_error / 4)

      print(pol_tr_errors)
      print(pol_val_errors)
```

```
[186993.19739953475, 185244.43010785463, 188080.52061730355, 189909.0856310002,
180383.88726492785, 185833.18185037965, 224341.00039622546, 212672.49994590002,
240164.82455914808, 231779.85671004708]
[187346.17569095048, 187119.67651227498, 196407.7523358208, 217105.58648126366,
```

298195.4819408711, 375210.469116806, 642143.0810408428, 2235337.509071591,
13428525.564918954, 5589924.7339983685]

```
[15]: poly = PolynomialFeatures(degree=2)      #the second degree seems to be the best
      ↪for predicting the price
test_error = 0
for i, (train, val) in enumerate(kfold.split(X_folds)):
    X_train, y_train = X_folds[train], y_folds[train]
    X_val, y_val = X_folds[val], y_folds[val]

    X_train_poly = poly.fit_transform(X_train)
    lin_regr.fit(X_train_poly, y_train)
    y_train_pred = lin_regr.predict(X_train_poly)

    X_test_poly = poly.fit_transform(X_test)      #testing the data
    y_test_pred = lin_regr.predict(X_test_poly)
    test_error += mean_absolute_error(y_test, y_test_pred)
test_error = test_error/4
print(test_error)
```

172307.04621519067

```
[16]: mlp_tr_errors = []
      mlp_val_errors = []

      for layers in range(1,11):                  #choosing
      ↪the number of layers

          mlp_regr = MLPRegressor(hidden_layer_sizes= tuple([15]*layers),
      ↪max_iter=1000, random_state=101682945)
          tr_error = 0.0
          val_error = 0.0

          for i, (train, val) in enumerate(kfold.split(X_folds)):      #KFold
      ↪validation and choosing validation set and training set
              X_train, y_train = X_folds[train], y_folds[train]
              X_val, y_val = X_folds[val], y_folds[val]

              mlp_regr.fit(X_train, y_train)      #Training
      ↪the model
              y_train_pred = mlp_regr.predict(X_train)
              tr_error += mean_absolute_error(y_train, y_train_pred)

              y_val_pred = mlp_regr.predict(X_val)
      ↪#Validationg the model
              val_error += mean_absolute_error(y_val, y_val_pred)
```

```

    mlp_tr_errors.append(tr_error / 4)                                #Average
    ↪of the error estimate from KFold validation
    mlp_val_errors.append(val_error / 4)

print(mlp_tr_errors)
print(mlp_val_errors)

```

```

[191040.31009567695, 191283.2001665017, 191211.50278291584, 191203.07282203954,
191170.87327096862, 191080.78236644118, 190154.24992993142, 191602.46420388803,
190891.95492476044, 190910.83173827338]
[191077.13719205477, 191307.13527856505, 191228.1074322185, 191165.15457427327,
191130.90262441378, 191115.0403882437, 190161.2150778934, 191629.0544386925,
190823.47905611517, 190905.5817064396]

```

```

[17]: mlp_regr = MLPRegressor(hidden_layer_sizes= tuple([15]*7), max_iter=1000,
    ↪random_state=101682945)    #the second degree seems to be the best for
    ↪predicting the price
test_error = 0
for i, (train, val) in enumerate(kfold.split(X_folds)):
    X_train, y_train = X_folds[train], y_folds[train]
    X_val, y_val = X_folds[val], y_folds[val]

    mlp_regr.fit(X_train, y_train)
    y_train_pred = mlp_regr.predict(X_train)

    y_test_pred = mlp_regr.predict(X_test)                                #testing
    ↪the data
    test_error += mean_absolute_error(y_test, y_test_pred)
test_error = test_error/4
print(test_error)

```

```

178359.6982392302

```

```

[ ]:

```